

MODULE 09

Python Agent SDK 源码逐模块教程

v0.2.141: public API、transport、control
protocol、sessions、MCP、错误与可观测性

Python SDK 是目前最可验证的官方开源入口。它不重写 Claude Code，而是把原生 CLI 作为子进程，通过 NDJSON 与双向 control protocol 暴露成 typed async API。

研究基线：Claude Code v2.1.236（稳定版） | Python Agent SDK v0.2.141（tag） | 历史源码观察 v2.1.88
资料核验日期：2026-08-19 | 中文深度研究版

目录

- 1. 源码树与职责
- 2. `query()`：最薄的一层
- 3. `ClaudeSDKClient`：双向生命周期
- 4. `InternalClient`：把各层接起来
- 5. `SubprocessCLITransport`：命令编译器 + 进程监护
- 6. `Query`：协议路由器
- 7. 权限 callback 与 hooks
- 8. Message parser 与前向兼容
- 9. Options：一张可编程控制面
- 10. SessionStore 与恢复
- 11. In-process MCP
- 12. 可观测性与成本
- 13. 错误、背压与取消
- 14. 生产封装模板

阅读提示：内部模块名以历史源码观察解释，当前功能以官方文档与 v0.2.141 SDK 源码校验。

1. 源码树与职责

模块	行数级别	职责
query.py	~127	one-shot async generator facade
client.py	~598	双向 <code>ClaudeSDKClient</code>
_internal/client.py	~197	组装 transport + Query + parser
_internal/query.py	~988	控制协议、读取循环、hooks、权限、MCP
transport/subprocess_cli.py	~1190	找 CLI、编 argv、spawn、NDJSON、清理
message_parser.py	~396	wire dict → typed messages
types.py	~2467	options、messages、hooks、permissions、session protocols
sessions.py	~2025	本地 transcript 查找、链重建、store reader
session_resume.py	~624	store → 临时 CLI config materialization
sdk_mcp_bridge.py	~385	in-process MCP session bridge

行数来自 2026-08-19 读取的 tag v0.2.141 附近文件，后续会变化；用途是显示复杂度重心，不是 API 承诺。 [S1]

2. query()：最薄的一层

Public `query()` 只做三件事：options 默认化、创建 `InternalClient`、转发 `process_query()` 产生的 typed messages。它适合一次性/单向流，不能在运行中 interrupt 或发 follow-up。 [S1]

示例 | 只读、有限轮次的一次性查询

```
import anyio
from claude_agent_sdk import ClaudeAgentOptions, ResultMessage, query

async def main():
    opts = ClaudeAgentOptions(
        cwd="/repo",
        permission_mode="dontAsk",
        allowed_tools=["Read", "Grep", "Glob"],
        max_turns=8,
    )
    async for msg in query(prompt="Explain the request path.", options=opts):
        if isinstance(msg, ResultMessage):
            print(msg.subtype, msg.session_id, msg.total_cost_usd)

anyio.run(main)
```

3. ClaudeSDKClient：双向生命周期

- `connect()`：校验 SessionStore 组合，必要时 materialize resume，spawn CLI，initialize control channel。
- `query()`：连接后可反复写 user NDJSON frame。
- `receive_messages()` / `receive_response()`：消费 typed stream；后者直到当前 result。
- 运行时控制：interrupt、set_model、set_permission_mode、rewind_files、MCP status/toggle/reconnect、stop_task。
- `disconnect()`：先关闭 Query/transport，再清理临时恢复目录。 [S1, S2]

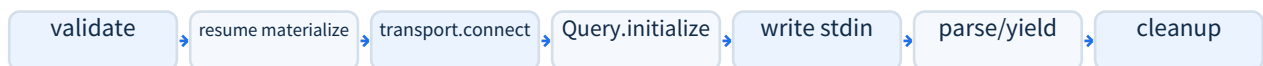
示例 | 交互式两阶段流程

```
async with ClaudeSDKClient(options=opts) as client:
    await client.query("Inspect failures, do not edit.")
    async for msg in client.receive_response():
        render(msg)
    await client.set_permission_mode("acceptEdits")
    await client.query("Apply the smallest verified fix.")
    async for msg in client.receive_response():
        render(msg)
```

运行时限制

同一个 client 必须留在创建它的 async runtime context；内部 read task/task group 从 connect 持续到 disconnect。跨 asyncio/trio context 复用会破坏 task affinity。[S1]

4. InternalClient: 把各层接起来



字符串 prompt 也走 streaming：先 initialize，再把 user frame 写入 stdin；这样 agents、hooks、skills filter 和 in-process MCP 都能通过 control protocol 工作。query() 看似单向，底层仍保留 stdin，直到结果与后台任务正确结清。[S1, S2]

finally 中显式关闭内部 generator，再删除临时配置目录。这段代码直接体现资源所有权：InternalClient 拥有 transport/Query/materialized resume，因此任何退出路径都必须释放。[S1]

5. SubprocessCLITransport: 命令编译器 + 进程监护

函数	职责
_find_cli	优先 bundled，回退 PATH；平台安全校验
_build_settings_value	把 sandbox 合并到 settings JSON
_apply_skills_defaults	skill filter 推导 allowedTools/setting sources
_build_command	options → argv；固定 stream-json 输入
connect	版本探测、env/OTel、spawn、建立 streams
_read_messages_impl	chunk → line frame → JSON dict；限制单消息大小
write/end_input	写锁保护 stdin
close	graceful → terminate → kill → reap

LineFramer 很关键：操作系统 pipe 提供任意 chunk，不保证一读一行。stdout/stderr 都要跨 chunk 拼行；单条 JSON 超过 `max_buffer_size` 必须硬失败，尾部半截 JSON 在 EOF 时记录并丢弃。[S3]

安全细节包括：optional-value flag 使用 equals form、防 Windows batch shim/路径歧义、过滤嵌套标志环境、显式版本最低要求、stderr callback 隔离和 child reaper。[S3]

6. Query：协议路由器

内部状态	作用
pending_control_responses/results	request_id → Event/结果
hook_callbacks	callback ID → Python callable
memory object stream(100)	解耦 read loop 与消费方，提供背压
_read_task/_child_tasks	统一取消所有后台协程
_inflight_requests	取消 control callback
_inflight_tasks	防止 turn result 过早关闭 stdin
_last_error_result	把尾随 code 1 提升为 ResultError
_transcript_mirror_batcher	result 前 flush 外部 SessionStore

`_read_messages()` 是路由中心：control_response 唤醒 waiter；control_request 派生 handler；cancel request 取消 handler；transcript_mirror 进入 batcher；system frame 更新 task ledger；result 决定 flush/close/error state；普通消息进入内存 stream。 [S2]

7. 权限 callback 与 hooks

CLI 发 `can_use_tool` 后，SDK 构造 `ToolPermissionContext`，包含 suggestions、tool_use_id、agent_id、blocked_path、decision_reason 和显示 metadata。callback 返回 Allow（可替换 input、更新 permissions）或 Deny（message、interrupt）。 [S2, S5]

示例 | 只对 ask 调用生效的 SDK permission callback

```
async def can_use_tool(name, input, ctx):
    if name in {"Read", "Grep", "Glob"}:
        return PermissionResultAllow()
    return PermissionResultDeny(
        message=f"{name} requires an interactive approval", interrupt=False
    )
```

Hooks 在 initialize 阶段变成 callback IDs。CLI 触发 `hook_callback` 时 SDK 找到 callable，await 结果，把 Python 安全字段 `async_/continue_` 转为 CLI 期望字段。 [S2]

8. Message parser 与前向兼容

wire type	typed object
user	UserMessage + text/tool_use/tool_result blocks
assistant	AssistantMessage + thinking/server tool blocks
system task_*	TaskStarted/Progress/Notification/Updated
result	ResultMessage + usage/cost/errors/origin
stream_event	StreamEvent
rate_limit_event	RateLimitEvent
conversation_reset	ConversationResetMessage
unknown	debug log + skip

对未知 top-level message type 选择 skip，而不是崩溃，是 SDK 与更快更新 CLI 共存的前向兼容策略；但已识别类型缺少必需字段则抛 `MessageParseError`，避免静默产生错误对象。origin dict 保留未知附加键，也是兼容设计。

[S4]

9. Options：一张可编程控制面

类别	代表字段
模型	model, fallback_model, thinking, effort, betas
工具	tools, allowed_tools, disallowed_tools, skills
权限	permission_mode, can_use_tool, hooks, sandbox
上下文	system_prompt, setting_sources, cwd, add_dirs
扩展	mcp_servers, strict_mcp_config, plugins, agents
停止	max_turns, max_budget_usd, task_budget
会话	resume, continue, fork, resume_session_at
输出	partial messages, hook events, JSON schema
持久化	checkpointing, SessionStore, flush mode

几个容易误解的字段：`allowed_tools` 不是 availability allowlist；`disallowed_tools` bare name 会移出 context；`skills=None` 不等于关闭全部 skills；`setting_sources=[]` 才是 SDK isolation mode；structured output 可能在最后 assistant message 之后还有条目，截断 resume 不可只盯 assistant UUID。

[S5]

10. SessionStore 与恢复

设置 `session_store` 后，CLI 用 `--session-mirror` 输出 mirror frames。Batcher 在每个 result 前 flush，因此调用方观察到 result 时可假定本 turn 已提交到 store——除非收到 `mirror_error`。

[S3, S8]

远端恢复不是直接把 entries 喂给模型，而是 materialize 成 CLI 原生目录结构；复制凭据/Keychain fallback、settings 与 subagent sidecars；过滤会导致空 plugin cache 重装的配置；finally 删除临时目录。

[S7]

11. In-process MCP

`McpSdkServerConfig` 把 Python `mcp.server.Server` 实例放入 options；InternalClient 从配置剥离 instance，只把可序列化部分交给 CLI，同时为实例创建 bridge。CLI 遇到该 server 的消息时通过 control channel 转给 SDK。

[S1, S2, S9]

每个 bridge session 维护 pending response、background runner 和 close 语义。调用方必须保证 tool handler 幂等/可取消；interrupt 在 MCP 2.x 可传播，但外部不可取消 I/O 仍需自己的超时与幂等策略。

[S9]

12. 可观测性与成本

CLI 内建 OTel；SDK 只把配置和 W3C trace context 传给子进程。CLI 直接向 collector 发 spans/metrics/log events。`ResultMessage.usage` 只覆盖主 loop；whole-tree 成本要看 `model_usage`，尤其有 subagents 时。

[O26, O27, S3, S4]

- 不要在 telemetry 中默认记录完整 prompt/tool output；按数据分类开关。

- span 关联 session_id、tool_use_id、task_id，但不要把标题当稳定 key。
- 记录 cache read/write、model、effort、permission decision、sandbox violation。
- 成本上限是停止机制，不是精确预算器；并行请求可能有在途用量。

13. 错误、背压与取消

- 消息内存流大小 100：消费方过慢会对 read loop 施加背压；UI 不应阻塞渲染线程。
- 单条 stdout JSON 有 max buffer；不是总会话上限。
- control request 有 timeout；初始化至少 60 秒以容纳 MCP 连接。
- ResultError 优先于泛化 ProcessError；调用方按 subtype 分支。
- close 使用 shield + 有界等待；raw asyncio cancel 仍有已知边界。
- mirror timeout 不盲重试；远端写入状态可能不确定。 [S2, S3, S8]

14. 生产封装模板

示例 | 示例：有限权限、有限成本、完整消费消息流

```
async def run_agent(prompt: str, *, repo: str):
    opts = ClaudeAgentOptions(
        cwd=repo,
        permission_mode="dontAsk",
        allowed_tools=["Read", "Grep", "Glob", "Bash(git status)"],
        disallowed_tools=["Bash(git push *)"],
        max_turns=20,
        max_budget_usd=5.0,
        include_hook_events=True,
        enable_file_checkpointing=True,
    )
    result = None
    async for msg in query(prompt=prompt, options=opts):
        await persist_event(msg)
        if isinstance(msg, ResultMessage):
            result = msg
    if result is None or result.subtype != "success":
        raise AgentRunFailed(result)
    return result
```

最后的验证

生产 wrapper 应测试：CLI 缺失、版本过低、stdout 半截 JSON、callback 抛错、hook 超时、MCP 断连、用户取消、background task 在 result 后完成、SessionStore 超时与进程 SIGTERM/SIGKILL。happy path 只覆盖最容易的一小部分。

参考资料与证据等级

[O23] 官方 | [Agent SDK overview](#)

<https://code.claude.com/docs/en/agent-sdk/overview>

[O24] 官方 | [Agent SDK agent loop](#)

<https://code.claude.com/docs/en/agent-sdk/agent-loop>

[O25] 官方 | [Python Agent SDK reference](#)

<https://code.claude.com/docs/en/agent-sdk/python>

[O26] 官方 | [Agent SDK observability](#)

<https://code.claude.com/docs/en/agent-sdk/observability>

[O27] 官方 | [Agent SDK cost tracking](#)

<https://code.claude.com/docs/en/agent-sdk/cost-tracking>

[S1] 开源源码 | [Python Agent SDK v0.2.141](#)

<https://github.com/anthropics/claude-agent-sdk-python/tree/v0.2.141>

[S2] 开源源码 | [Internal Query control protocol](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/query.py

[S3] 开源源码 | [Subprocess CLI transport](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/transport/subprocess_cli.py

[S4] 开源源码 | [Message parser](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/message_parser.py

[S5] 开源源码 | [SDK types and options](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/types.py

[S6] 开源源码 | [Session reader](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/sessions.py

[S7] 开源源码 | [Session resume materialization](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/session_resume.py

[S8] 开源源码 | [Transcript mirror batcher](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/transcript_mirror_batcher.py

[S9] 开源源码 | [In-process SDK MCP bridge](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/sdk_mcp_bridge.py

方法说明

官方材料用于确认当前稳定行为；SDK 源码固定在发布 tag v0.2.141；社区材料只用于解释 v2.1.88 历史源码结构。源码行号和内部函数名可能在后续版本变化，外部契约以官方最新文档为准。